IMS BACKEND

Project Lead Handbook

Team Allocation · API Contracts · Dev Instructions · Timeline

Stack: MongoDB · Express.js · React.js · Node.js | Target: 1–2 Days

1

Branch & PR Rules — Communicate to Team

Rule	Detail
Branch naming	feature/{your-name}/{module} e.g. feature/riya/auth
No direct pushes	main is protected. Every change goes through a PR.
PR requirement	Your PR must have: description, list of endpoints added, any env vars added.
Merge approval	Project lead (you) reviews and merges all PRs.
Conflict rule	If you change a shared file (constant.js, enums.js, customError.js), flag it in the group chat first.
Daily sync	Short 15-min call at start of day. Each dev says: done yesterday, doing today, any blockers.

2 API Contracts — Complete Reference

This is the single source of truth for all API endpoints. Every developer must match these contracts exactly — method, path, request body field names, and response shape. Do not deviate. If you need a field that is not here, raise it with the project lead before adding it.

Base URL: All endpoints are prefixed with /api/v1. Auth header for protected routes: Authorization: Bearer {token}

2.1 Auth Endpoints

Method	Endpoint	Auth	Description
POST	/api/v1/auth/login	Public	Login with email + password. Returns access + refresh token.
POST	/api/v1/auth/logout	JWT	Invalidates refresh token.
POST	/api/v1/auth/refresh	Public	Exchange refresh token for new access token.
POST	/api/v1/auth/change-password	JWT	Change own password. Body: { currentPassword, newPassword }.

POST /auth/login — Request

```
{ "email": "string", "password": "string" }
```

POST /auth/login — Response 200

```
{
  "success": true,
  "accessToken": "string",
  "refreshToken": "string",
  "user": { "id": "string", "name": "string", "role": "admin|teacher|student", "email":
"string" }
}
```

2.2 Admin — User Management

Method	Endpoint	Auth	Description
GET	/api/v1/admin/dashboard	Admin	Summary: total batches, students, teachers, top/bottom scorers.
POST	/api/v1/admin/teachers	Admin	Create teacher account.
GET	/api/v1/admin/teachers	Admin	List all teachers.
PUT	/api/v1/admin/teachers/:id	Admin	Edit teacher profile.
PATCH	/api/v1/admin/teachers/:id/status	Admin	Toggle active/inactive. Body: { active: boolean }.
POST	/api/v1/admin/students	Admin	Create single student account.
POST	/api/v1/admin/students/bulk	Admin	Bulk create via CSV upload. Form-data: file (text/csv).

GET	`/api/v1/admin/students`	Admin	List all students. Query: ?batchId=, ?active=.
PATCH	`/api/v1/admin/students/:id/status`	Admin	Toggle active/inactive.
PATCH	`/api/v1/admin/students/:id/batch`	Admin	Move student to different batch. Body: { newBatchId }.

POST /admin/teachers — Request

```
{
  "name": "string",
  "email": "string",
  "password": "string",
  "designation": "string",
  "bio": "string"
}
```

POST /admin/students — Request

```
{
  "name": "string",
  "email": "string",
  "password": "string",
  "batchId": "string",
  "enrollmentDate": "ISO date string"
}
```

2.3 Admin — Batch Management [ON HOLD Not to be developed]

Method	Endpoint	Auth	Description
POST	`/api/v1/admin/batches`	Admin	Create batch.
GET	`/api/v1/admin/batches`	Admin	List all batches.
PUT	`/api/v1/admin/batches/:id`	Admin	Edit batch name, dates, assigned teachers.
PATCH	`/api/v1/admin/batches/:id/close`	Admin	Close batch. No body required.
POST	`/api/v1/admin/batches/:id/recruiter-link`	Admin	Generate recruiter link. Returns { recruiterUrl }.
DELETE	`/api/v1/admin/batches/:id/recruiter-link`	Admin	Revoke recruiter link. Returns 204.
GET	`/api/v1/admin/batch-config/:batchId`	Admin	Get current mark params for batch.
PUT	`/api/v1/admin/batch-config/:batchId`	Admin	Update mark params for batch.

POST /admin/batches — Request

```
{
  "name": "string",
  "description": "string",
  "startDate": "ISO date string",
  "endDate": "ISO date string",
  "teacherIds": ["string"],
  "studentIds": ["string"]
```

```
}
```

PUT /admin/batch-config/:batchId — Request

```
{
  "baseScore": 30,
  "attendanceFull": 2.5,
  "attendanceHalf": 1.0,
  "attendanceMissed": -5.0,
  "quizMax": 5,
  "quizMissed": -2.5,
  "markCap": 100,
  "reason": "string (min 20 chars)"
}
```

2.4 Admin — Mark Overrides & Audit [ON HOLD Not to be developed]

Method	Endpoint	Auth	Description
POST	/api/v1/admin/marks/override	Admin	Adjust student total marks. Body below.
POST	/api/v1/admin/marks/event-correction	Admin	Correct specific ledger event. Body below.
POST	/api/v1/admin/marks/manual-score	Admin	Enter manual score for errored code review.
POST	/api/v1/admin/marks/recalculate/:studentId	Admin	Trigger full recalculation for one student.
POST	/api/v1/admin/marks/recalculate-batch/:batchId	Admin	Trigger full recalculation for entire batch.
GET	/api/v1/admin/audit-log	Admin	Paginated audit trail. Query: ?from=&to=&adminId=&actionType=&page=&limit=
GET	/api/v1/admin/audit-log/export	Admin	Download audit trail as CSV file.

POST /admin/marks/override — Request

```
{
  "studentId": "string",
  "delta": "number (positive or negative)",
  "reason": "string (min 20 chars)"
}
```

POST /admin/marks/event-correction — Request

```
{
  "studentId": "string",
  "ledgerEventId": "string",
  "newMark": "number",
  "reason": "string (min 20 chars)"
}
```

POST /admin/marks/manual-score — Request

```
{
  "submissionId": "string",
  "manualScore": "number (0–10)",
  "reason": "string (min 20 chars)"
}
```

2.5 Teacher Endpoints

Method	Endpoint	Auth	Description
GET	/api/v1/teacher/dashboard	Teacher	Upcoming lectures, pending CSV uploads.
PUT	/api/v1/teacher/profile	Teacher	Edit own name, designation, bio. Photo via multipart.
GET	/api/v1/teacher/batches	Teacher	List batches assigned to this teacher.
GET	/api/v1/teacher/students/:batchId	Teacher	Student mark breakdown for a batch.
GET	/api/v1/teacher/lecture-summary/:lectureId	Teacher	Attendance count, avg quiz, avg assignment for a lecture.

2.6 Curriculum / Topics

Method	Endpoint	Auth	Description
GET	/api/v1/teacher/topics/:batchId	Teacher	List all topics in curriculum order.
POST	/api/v1/teacher/topics	Teacher	Create topic.
PUT	/api/v1/teacher/topics/:id	Teacher	Edit topic.
DELETE	/api/v1/teacher/topics/:id	Teacher	Delete topic (blocked if lecture linked).
PATCH	/api/v1/teacher/topics/reorder	Teacher	Reorder topics. Body: { topicIds: [] } in new order.
POST	/api/v1/teacher/topics/:id/notes	Teacher	Upload notes file. Multipart form-data.
DELETE	/api/v1/teacher/topics/:id/notes/:fileId	Teacher	Delete a notes file.

POST /teacher/topics — Request

```
{
  "batchId": "string",
  "title": "string (max 120 chars)",
  "description": "string (HTML)",
  "learningObjectives": ["string"],
  "estimatedHours": "number"
}
```

2.7 Lecture Management

Method	Endpoint	Auth	Description
GET	/api/v1/teacher/lectures/:batchId	Teacher	List all lectures for a batch.
POST	/api/v1/teacher/lectures	Teacher	Create lecture.

PUT	/api/v1/teacher/lectures/:id	Teacher	Edit lecture (Scheduled status only).
PATCH	/api/v1/teacher/lectures/:id/status	Teacher	Transition status. Body: { status }.
DELETE	/api/v1/teacher/lectures/:id	Teacher	Cancel lecture. Sets status = cancelled.

POST /teacher/lectures — Request

```
{
  "batchId": "string",
  "topicIds": ["string"],
  "title": "string",
  "date": "ISO date string",
  "startTime": "HH:MM",
  "endTime": "HH:MM",
  "half1EndTime": "HH:MM",
  "meetUrl": "string",
  "assignmentTitle": "string",
  "assignmentDescription": "string (HTML)",
  "assignmentDeadline": "ISO datetime string",
  "githubRepoSeed": "string (optional)"
}
```

PATCH /teacher/lectures/:id/status — Request

```
{ "status": "in_progress | completed | cancelled" }
```

2.8 Attendance & Quiz CSV Upload

Method	Endpoint	Auth	Description
POST	/api/v1/teacher/lectures/:id/attendance	Teacher	Upload attendance CSV. Multipart: file (text/csv).
GET	/api/v1/teacher/lectures/:id/attendance	Teacher	Get processed attendance for a lecture.
POST	/api/v1/teacher/lectures/:id/quiz	Teacher	Upload quiz score CSV. Multipart: file (text/csv).
GET	/api/v1/teacher/lectures/:id/quiz	Teacher	Get processed quiz scores for a lecture.

csv/payload example :
name,first_half,Second_half
John,true,true
Smith,false,false
abc,true,false

POST .../attendance — Response 200

```
{
  "success": true,
  "processed": 28,
  "errors": [],
  "summary": [
    { "studentId": "string", "email": "string", "halfAttended": "both", "marksApplied":
2.5, "newTotal": 67.5 }
  ]
}
```

POST .../attendance — Response 400 (validation failure)

```
{
  "success": false,
  "errors": [
    { "row": 4, "field": "half_attended", "message": "Invalid value 'yes'. Must be
both|1|2|none" },
    { "row": 7, "field": "student_email", "message": "Email not found in batch" }
  ]
}
```

2.9 Student Endpoints

Method	Endpoint	Auth	Description
GET	/api/v1/student/dashboard	Student	Total marks, rank, upcoming lectures, pending assignments.
GET	/api/v1/student/marks/history	Student	Full mark ledger (paginated). Query: ?page=&limit=
PUT	/api/v1/student/profile	Student	Edit own name. Photo via multipart.
GET	/api/v1/student/curriculum	Student	Curriculum topics with completion status for own batch.
GET	/api/v1/student/assignments	Student	List all published + closed assignments. Includes submission status.
GET	/api/v1/student/assignments/:id	Student	Single assignment detail.
POST	/api/v1/student/assignments/:id/submit	Student	Submit GitHub URL. Body: { githubUrl }.
GET	/api/v1/student/assignments/:id/review	Student	Code review result + feedback for own submission.
GET	/api/v1/student/portfolio	Student	Own full portfolio with all metrics.
GET	/api/v1/student/portfolio/pdf	Student	Stream PDF export of own portfolio.
GET	/api/v1/student/notifications	Student	Paginated in-app notifications.
PATCH	/api/v1/student/notifications/:id/read	Student	Mark single notification read.
PATCH	/api/v1/student/notifications/read-all	Student	Mark all notifications read.

GET /student/dashboard — Response 200

```
{
  "totalMarks": 74.5,
  "batchRank": 3,
  "batchSize": 22,
  "upcomingLectures": [ { "id": "string", "title": "string", "date": "ISO", "meetUrl":
"string" } ],
  "pendingAssignments": [ { "id": "string", "title": "string", "deadline": "ISO",
"hoursLeft": 18.5 } ],
  "unreadNotifications": 2
}
```

POST /student/assignments/:id/submit — Request

```
{ "githubUrl": "https://github.com/{user}/{repo}" }
```

POST /student/assignments/:id/submit — Response 200

```
{
  "success": true,
  "submissionId": "string",
  "isLate": false,
  "reviewStatus": "pending",
  "message": "Submission received. Code review in progress."
}
```

2.10 Recruiter Endpoints (No Auth)

Method	Endpoint	Auth	Description
GET	`/api/v1/recruiter/:batchUuid`	Public	Batch overview + teacher info + curriculum summary.
GET	`/api/v1/recruiter/:batchUuid/students`	Public	Sortable/searchable student list with summary metrics. Query: ?sort=totalMarks&order=desc&search=name
GET	`/api/v1/recruiter/:batchUuid/students/:id`	Public	Full portfolio for one student with all metrics.

GET /recruiter/:batchUuid/students — Response 200

```
{
  "batchName": "string",
  "students": [
    {
      "id": "string",
      "name": "string",
      "photo": "url | null",
      "totalMarks": 74.5,
      "batchRank": 3,
      "attendanceRate": 88.0,
      "onTimeSubmissionRate": 100.0,
      "engagementScore": 82.4,
      "consistencyScore": 76.0
    }
  ],
  "total": 22
}
```

> **Caching Rule:** Recruiter endpoints must cache computed metrics per batch with a 5-minute TTL using Redis. Invalidate cache on any new ledger event for that batch.

2.11 Standard Response Envelope

Every endpoint — success or error — must return a JSON response in this exact shape. No deviations.

Success

```
{ "success": true, "data": {}, "message": "optional string" }
```

Error

```
{ "success": false, "message": "human-readable error", "errors": [] }
```

HTTP Status Codes to Use

Code	When to Use
200	Successful GET, PUT, PATCH
201	Successful POST that creates a resource
204	Successful DELETE with no response body
400	Validation error, bad request body, CSV rejection
401	Missing or invalid JWT token
403	Valid token but wrong role
404	Resource not found (including revoked recruiter link)
409	Conflict — e.g. duplicate submission, email already exists
500	Unhandled server error (caught by global error handler)

DEV 1 — *Auth · Admin · Shared Middleware*

Your Modules

- Authentication (login, logout, refresh token, change password)
- JWT middleware — this is what EVERYONE imports. Get it right.
- Role-based access middleware (admin, teacher, student guards)
- Admin: user management (teachers + students)
- Admin: batch management + batch config
- Admin: mark overrides + manual score entry
- Admin: audit log (read + CSV export)
- Admin: recruiter link generate + revoke
- Admin: dashboard summary
- Admin: full recalculation trigger

File Ownership

File	Your Responsibility
src/controller/authController.js	Login, logout, refresh, changePassword
src/controller/adminController.js	All admin actions
src/middleware/auth.js	verifyToken, requireRole(role) — SHARED, everyone imports this
src/middleware/errorHandler.js	Global error handler — catches all thrown errors
src/models/User.js	User schema (all roles in one collection, role field distinguishes)
src/models/Batch.js	Batch schema including recruiterUuid, status, teacherIds, studentIds
src/models/BatchConfig.js	Per-batch mark parameters
src/models/AuditLog.js	Immutable audit entries
src/models/MarkLedger.js	CRITICAL — append-only mark event log. Every dev reads this.
src/service/authService.js	Business logic for auth
src/service/adminService.js	Business logic for admin actions
src/service/markService.js	recalculate(), applyMarkEvent(), reverseMarkEvent() — SHARED
src/routes/authRoutes.js	
src/routes/adminRoutes.js	

Priority Build Order

Order	Task	Why
1st	src/middleware/auth.js — verifyToken + requireRole	Team blocker. Push this alone as soon as it works.
2nd	User model + Batch model + MarkLedger model	All other devs need these models to exist.
3rd	Auth routes (login/logout/refresh)	Everyone needs to test their routes.
4th	markService.js — recalculate() + applyMarkEvent()	Dev 4 and Dev 5 both depend on this logic.
5th	Admin user + batch CRUD	Needed to create test data for other devs.
6th	Mark override, audit log, batch config, recruiter link	Finish your module fully.

Mark Ledger — Critical Notes

Rule: MarkLedger is append-only. NEVER update or delete a ledger document. To correct a mark, insert a REVERSAL entry (negative of original) and then a new CORRECTED entry. The running total is always computed by summing all ledger entries for a student.

MarkLedger Schema (define this exactly)

```
{
  studentId: ObjectId (ref: User),
  batchId: ObjectId (ref: Batch),
  lectureId: ObjectId (ref: Lecture, optional),
  eventType: enum (from enums.LEDGER_EVENT),
  marksApplied: Number,
  runningTotal: Number,    // stored for fast dashboard display
  isReversal: Boolean,     // true for correction reversal entries
  reversalOf: ObjectId,    // points to the original ledger entry being reversed
  meta: Object,            // { csvId, submissionId, auditId — whichever is relevant }
  createdAt: Date
}
```

markService.js — Functions You Must Export

```
// Appends a new ledger event and updates running total
async function applyMarkEvent({ studentId, batchId, lectureId, eventType, marksApplied,
meta })

// Reverses a specific ledger entry and re-applies corrected value
async function reverseAndCorrect({ studentId, ledgerEventId, newMark, auditId })

// Full recalculation from scratch — recomputes runningTotal on all entries
async function recalculate(studentId)

// Full batch recalculation
async function recalculateBatch(batchId)
```

Middleware — What to Export from auth.js

```
// Usage in routes: router.get('/path', verifyToken, requireRole('admin'), controller)
module.exports = {
  verifyToken,        // Decodes JWT, attaches req.user = { id, role, email }
  requireRole,        // requireRole('admin') or requireRole(['admin','teacher'])
};
```

Recalculation Logic — Enforce This Exactly

- Fetch ALL ledger entries for the student in ascending createdAt order.
- Start from batchConfig.baseScore (NOT a hardcoded 30 — read from BatchConfig).
- Sum all marksApplied values.
- If running total exceeds batchConfig.markCap, store excess in a separate field and cap to markCap.
- Update runningTotal on each ledger entry (in memory only; write the final total to the User document's currentMarks field).
- Entire operation must be wrapped in a MongoDB session/transaction. Rollback on any failure.
- Log the recalculation event to AuditLog.

DEV 2 — *Lecture · Curriculum · Notifications · Code Review Queue*

Dependency: Wait for Dev 1 to push auth middleware and User/Batch models before starting routes. You can build your models and service logic in parallel while waiting.

Your Modules

- Curriculum topic CRUD (create, edit, delete, reorder)
- Notes file upload per topic
- Lecture CRUD and status transitions
- Assignment record auto-creation when lecture is marked Completed
- Notification triggers (schedule jobs for T−24h and T−1h reminders)
- Code review job queue (Bull) — submit URL, receive score + feedback, handle errors
- Code review retry logic and admin error alerts

File Ownership

File	Your Responsibility
src/models/Topic.js	Curriculum topic schema
src/models/Lecture.js	Lecture schema with status enum
src/models/Assignment.js	Assignment schema linked to lecture
src/models/Notification.js	In-app notification schema
src/controller/teacherController.js	All teacher-facing actions (topics, lectures, notes)
src/service/curriculumService.js	Topic CRUD business logic
src/service/lectureService.js	Lecture creation, status transitions, assignment publish
src/service/notificationService.js	createNotification(), sendEmail(), scheduleReminders()
src/service/codeReviewService.js	callCodeReviewApi(), handleReviewResult(), handleReviewError()
src/utils/cron/lectureReminders.js	Cron job or Bull delayed job for T−24h and T−1h notifications
src/routes/teacherRoutes.js	All /api/v1/teacher/* routes

Lecture Status Transition Rules — Enforce in Service

From	To	Allowed When	Side Effect
Scheduled	In Progress	Always (teacher triggers manually).	Record actual_start_time.
In Progress	Completed	Server timestamp >= lecture.endTime. Block + return 400 if too early.	Publish assignment. Trigger notification to all students.
Scheduled	Cancelled	Always.	Send cancellation notification. No marks applied.
In Progress	Cancelled	Always.	Same as above.
Completed	Any	BLOCKED. A completed lecture cannot be changed.	Return 400 with message.
Cancelled	Any	BLOCKED.	Return 400 with message.

Assignment Auto-Publish Logic

- When lecture status transitions to Completed, your service must automatically:

Create an Assignment document linked to the lecture with status = 'published'.
Set assignment.deadline from the lecture.assignmentDeadline field.
Create a Notification document for every student in the batch: type = 'assignment_published'.
Schedule a deadline-approaching job: fire at (deadline − 24 hours) for students who haven't submitted.

Code Review Job Queue — Architecture

How it works

- When a student submits a GitHub URL (Dev 3 triggers this), Dev 3 calls codeReviewService.queueReview(submissionId).
- Your queue worker picks up the job, calls the external code review API, gets { score, feedback }.
- On success: update Submission document, call markService.applyMarkEvent() with the score (or 0 if late).
- On failure: retry up to 3 times with delays from constants.REVIEW_RETRY_DELAYS. After 3 failures: set submission.reviewStatus = 'error', create admin notifications for all 4 admins.

Queue Setup

```
// src/service/codeReviewService.js
const Queue = require('bull');
const reviewQueue = new Queue('code-review', process.env.REDIS_URL);

// Enqueue a review job
async function queueReview(submissionId) {
  await reviewQueue.add({ submissionId }, { attempts: 3, backoff: { type: 'fixed', delay:
60000 } });
}

// Worker (runs in same or separate process)
reviewQueue.process(async (job) => {
  const { submissionId } = job.data;
  // 1. Fetch submission from DB
  // 2. Call CODE_REVIEW_API_URL with githubUrl
  // 3. On success: update submission, call markService.applyMarkEvent()
  // 4. Create student notification
});

reviewQueue.on('failed', async (job, err) => {
  // After all retries exhausted: set reviewStatus = 'error', alert all admins
});
```

Notification Service — What to Export

```
module.exports = {
  createNotification(userId, type, message, meta),  // saves to DB
  sendEmail(to, subject, html),                      // sends via email provider
  notifyBatch(batchId, type, message),               // creates notif for all students in
batch
  notifyAdmins(type, message),                       // creates notif for all 4 admins
  scheduleReminder(lectureId, fireAt, type),         // delayed Bull job for reminders
};
```

Important: Dev 4 (attendance) and Dev 3 (submissions) will both call your notificationService. Make sure it is exported and documented before they need it.

DEV 3 — *Student Dashboard · Assignment Submission · Portfolio PDF*

Dependency: You need Dev 1's auth middleware and User model. You need Dev 2's Assignment model and codeReviewService.queueReview(). Coordinate with Dev 2 on when Assignment model is ready.

Your Modules

- Student dashboard (marks summary, rank, upcoming lectures, pending assignments)
- Mark history ledger view (paginated)
- Assignment list and submission flow
- Code review result view
- Student portfolio view (reads metrics computed by Dev 5's service)
- Portfolio PDF export (server-side generation)
- Student profile edit
- Notification read/list

File Ownership

File	Your Responsibility
src/controller/studentController.js	All student-facing actions
src/models/Submission.js	Student assignment submission schema
src/service/studentService.js	Dashboard data aggregation, submission logic
src/service/pdfService.js	Server-side PDF generation for portfolio
src/routes/studentRoutes.js	All /api/v1/student/* routes

Submission Logic — Step by Step

Step	What You Do	Notes
1. Receive URL	Validate githubUrl matches regex: /^https:\/\/github\.com\/[\w.-]+\/[\w.-]+$/	Return 400 if invalid.
2. Check assignment	Fetch Assignment by id. Check status === 'published'.	Return 400 if closed or unpublished.
3. Check duplicate	Query Submission where assignmentId + studentId. If exists, return 409.	One submission per student per assignment.
4. Check deadline	Compare server Date.now() with assignment.deadline.	Set isLate = Date.now() > deadline.
5. Save submission	Create Submission: { assignmentId, studentId, githubUrl, submittedAt, isLate, reviewStatus: 'pending' }.	
6. Queue review	Call codeReviewService.queueReview(submission._id).	Dev 2 owns this function.
7. Return response	Return 201 with { submissionId, isLate, reviewStatus: 'pending' }.	

Submission Schema

```
{
  assignmentId: ObjectId (ref: Assignment),
  studentId: ObjectId (ref: User),
  githubUrl: String,
  submittedAt: Date,      // Date.now() at server receipt
  isLate: Boolean,
  reviewStatus: enum ['pending', 'completed', 'error'],
  reviewScore: Number,    // raw score from code review (even if late)
  reviewFeedback: String,
  markAwarded: Number,    // 0 if late, reviewScore if on time
  ledgerEventId: ObjectId // reference to the mark_ledger entry created
}
```

Dashboard — Data You Need to Aggregate

- currentMarks: read from User.currentMarks (Dev 1 keeps this updated via markService).
- batchRank: query MarkLedger, group by studentId, sum marksApplied, rank by total — OR use User.currentMarks if stored.
- upcomingLectures: query Lecture where batchId = student.batchId, date >= today, status = scheduled/in_progress, limit 5.
- pendingAssignments: query Assignment where batchId, status = published, AND no Submission exists for this student yet.
- hoursLeft: compute (assignment.deadline - Date.now()) / 3600000, round to 1 decimal.

PDF Export — Library & Rules

- Use pdfkit (npm install pdfkit) for server-side PDF generation.
- The PDF must include: student name, photo, total marks, batch rank, all metrics from Section 6 of the SRS, assignment history table, curriculum completion.
- Metrics are computed by Dev 5's portfolioMetricsService. Import and call it.
- Stream the PDF directly to the response: res.setHeader('Content-Type', 'application/pdf'); pdfDoc.pipe(res);
- Do not save the PDF to disk. Generate and stream on request.

DEV 4 — *Attendance · Quiz CSV · Recalculation*

Dependency: You need Dev 1's auth middleware, MarkLedger model, and markService.applyMarkEvent(). You also need Dev 2's Lecture model to validate that a lecture exists and has the right status before accepting a CSV upload.

Your Modules

- Attendance CSV upload, validation, and mark processing
- Attendance CSV re-upload with automatic recalculation
- Quiz CSV upload, validation, and mark processing
- Quiz CSV re-upload with automatic recalculation
- Per-lecture attendance and quiz result retrieval

File Ownership

File	Your Responsibility
src/controller/csvController.js	Attendance and quiz CSV upload endpoints
src/models/AttendanceCsv.js	Archive of uploaded attendance CSVs per lecture
src/models/QuizCsv.js	Archive of uploaded quiz CSVs per lecture
src/models/AttendanceRecord.js	Per-student attendance result per lecture
src/models/QuizRecord.js	Per-student quiz score per lecture
src/service/attendanceService.js	CSV parse, validate, apply marks, re-upload logic
src/service/quizService.js	CSV parse, validate, apply quiz marks, re-upload logic
src/validator/csvValidator.js	Row-level validation logic for both CSV types
src/routes/csvRoutes.js	Mounted under /api/v1/teacher/lectures/:id/

Attendance CSV Processing — Exact Algorithm

Step	Action	Error Behaviour
1	Receive file. Check MIME type is text/csv. Check file size ≤ 5 MB.	Return 400 if invalid.
2	Parse CSV using a library (csv-parse npm). Check headers: student_email and half_attended both present.	Return 400 with missing header details.
3	Iterate every row. Validate: email exists in batch, half_attended is one of: both/1/2/none (case-insensitive).	Collect ALL row errors. Reject entire file. Return 400 with full errors array.
4	Check: is there an existing AttendanceCsv for this lecture? If yes — RECALCULATION MODE (see below).	
5 (new)	For each CSV row: compute marksApplied (from BatchConfig, not hardcoded constants). Call markService.applyMarkEvent().	Use MongoDB transaction wrapping all writes.
6 (new)	For each student in batch NOT in CSV: record as 'none', call markService.applyMarkEvent()	

	with attendance_missed value from BatchConfig.	
7 (new)	Save AttendanceCsv archive document. Save AttendanceRecord per student.	Rollback everything on failure.
8	Return 200 with per-student summary.	

Recalculation Mode — When CSV is Re-uploaded

- Fetch all MarkLedger entries for this lecture with eventType = 'attendance'.
- For each: call markService.reverseAndCorrect() — this inserts a REVERSAL ledger entry.
- Then process the new CSV from Step 5 above (apply new marks).
- Create AuditLog entry: { actionType: 'CSV_REPLACE', targetType: 'lecture', targetId: lectureId, oldValue: { previousCsvId, marksPerStudent }, newValue: { newCsvId, marksPerStudent } }.
- Entire operation (reversals + new marks) in a single MongoDB session/transaction.

Quiz CSV Processing — Same Pattern, Different Rules

Rule	Detail
Required columns	student_email, score
Score validation	Numeric. 0 ≤ score ≤ 5. Decimal allowed. Score > 5 → reject ENTIRE CSV.
Missing students	Students in batch not in CSV → score recorded as quiz_missed, deduction from BatchConfig.quizMissed.
Zero score	Score = 0 is valid. No deduction. +0 marks. Do NOT treat as absent.
One quiz per lecture	If QuizCsv already exists for this lecture: RECALCULATION MODE (same as attendance).
No quiz = no penalty	If teacher never uploads a quiz CSV for a lecture, no quiz marks are applied. No penalty.

Mark Application — Exactly How to Call markService

```
// Example: applying attendance marks after CSV processing
await markService.applyMarkEvent({
  studentId: student._id,
  batchId: lecture.batchId,
  lectureId: lecture._id,
  eventType: enums.LEDGER_EVENT.ATTENDANCE,
  marksApplied: computedMark,   // from BatchConfig, not hardcoded
  meta: { csvId: attendanceCsv._id, halfAttended: row.half_attended }
});
```

Reminder: NEVER import constants.js values for mark amounts inside your CSV service. Always fetch the batch's BatchConfig document and read the values from there. This ensures admin-configured custom values are respected.

DEV 5 — *Recruiter Portfolio · Metrics Engine · Caching*

Dependency: You depend on nearly everyone. You need: Dev 1 (Batch, MarkLedger, User models), Dev 3 (Submission model), Dev 4 (AttendanceRecord, QuizRecord models). Build your metric functions early using mock data, then wire up real models as they become available.

Your Modules

- Recruiter batch overview endpoint (no auth)
- Recruiter student list with sort + search
- Recruiter individual student portfolio
- All 18 metric computation functions (the full metrics engine)
- Redis caching layer for recruiter endpoints (TTL 5 min)
- Batch UUID lookup and revocation check

File Ownership

File	Your Responsibility
src/controller/recruiterController.js	Recruiter-facing endpoints (no auth middleware on these)
src/service/metricsService.js	All 18 metric computation functions — EXPORTED for Dev 3 (PDF) to import
src/service/cacheService.js	getBatchMetricsCache(), setBatchMetricsCache(), invalidateBatchCache()
src/routes/recruiterRoutes.js	All /api/v1/recruiter/* routes — NO verifyToken on these

Metrics Engine — All 18 Functions

Build each as a standalone async function that takes studentId (and optionally batchId) and returns a single computed value. This makes them testable individually and importable by Dev 3.

Function Name	Inputs	Returns	Formula
getTotalScore	studentId	number	Sum all ledger entries + baseScore, cap at markCap.
getBatchRank	studentId, batchId	number	Position by descending total score in batch.
getBatchPercentile	studentId, batchId	number	% of students scoring below this student.
getAssignmentAvgScore	studentId	number	Mean of markAwarded across all completed reviews.
getQuizAvgScore	studentId	number	Mean of quiz scores (participated quizzes only, i.e., score ≥ 0 and not quiz_missed event).
getQuizParticipationRate	studentId, batchId	number	(participated / total scheduled) × 100.
getAttendanceRate	studentId, batchId	number	(full-attendance lectures / total) × 100.
getOnTimeSubmissionRate	studentId, batchId	number	(on-time submissions / total published assignments) × 100.
getPunctualityIndex	studentId, batchId	number	Mean of (full=1.0, half=0.4, absent=0) per lecture × 100.

getSubmissionLeadTime	studentId	number	Mean hours between submittedAt and deadline for on-time subs.
getZeroMissStreaks	studentId, batchId	number	Longest consecutive run of full-attendance lectures.
getCodeQualityAvg	studentId	number	Mean of reviewScore (all completed reviews, including late).
getCodeImprovementRate	studentId	number	Linear regression slope of reviewScore over assignment sequence.
getPerfectAssignmentCount	studentId	number	Count of assignments where reviewScore = 10.
getBelowAvgAssignmentRate	studentId, batchId	number	(assignments below batch mean score / total) × 100.
getConsistencyScore	studentId	number	max(0, 100 − (SD of per-lecture total event scores × 10)).
getGrowthRate	studentId, batchId	number	Mean score (last 30% lectures) − Mean score (first 30% lectures).
getEngagementScore	studentId, batchId	number	(quizParticipation×0.3 + onTimeSubmission×0.4 + attendanceRate×0.3) scaled to 100.

Metrics Service — Export Shape

```
// src/service/metricsService.js
module.exports = {
  getTotalScore,
  getBatchRank,
  getBatchPercentile,
  getAssignmentAvgScore,
  getQuizAvgScore,
  getQuizParticipationRate,
  getAttendanceRate,
  getOnTimeSubmissionRate,
  getPunctualityIndex,
  getSubmissionLeadTime,
  getZeroMissStreaks,
  getCodeQualityAvg,
  getCodeImprovementRate,
  getPerfectAssignmentCount,
  getBelowAvgAssignmentRate,
  getConsistencyScore,
  getGrowthRate,
  getEngagementScore,

  // Convenience: compute all metrics for one student at once
  async getAllMetrics(studentId, batchId) { ... }
};
```

Caching Strategy

- Use Redis (ioredis) for caching. The key pattern is: metrics:batch:{batchId}:student:{studentId}
- Cache TTL: 300 seconds (5 minutes). Read from constants.METRIC_CACHE_TTL.
- On recruiter page load: check cache first. If hit → return cached. If miss → compute, store, return.
- Cache invalidation: whenever a new MarkLedger entry is written for any student in the batch, invalidate that student's cache key. Dev 1's markService.applyMarkEvent() must call cacheService.invalidateBatchCache(batchId, studentId) after every write.

cacheService.js — Export These

```
module.exports = {
  async getStudentMetricsCache(batchId, studentId),     // returns parsed object or null
  async setStudentMetricsCache(batchId, studentId, data), // serialises + sets TTL
  async invalidateStudentCache(batchId, studentId),      // deletes key
  async invalidateBatchCache(batchId),                   // deletes all keys for batch
(use pattern delete)
};
```

Recruiter UUID Lookup

- Recruiter routes receive a batchUuid (not the internal _id). Query Batch where recruiterUuid = batchUuid and recruiterLinkActive = true.
- If no batch found OR recruiterLinkActive = false → return HTTP 404 with message: 'This link is no longer active.'
- Never expose the internal batch _id in recruiter responses. Use recruiterUuid throughout.

No Auth on Recruiter Routes: Do NOT add verifyToken or requireRole to any /api/v1/recruiter/* route. These are intentionally public. Just add the batchUuid lookup as a middleware on the recruiter router.

8 Cross-Cutting Rules — Every Developer Must Follow

Error Handling

- Wrap ALL async route handlers in a try/catch. In the catch block: throw a CustomError or pass to next(err).
- The global error handler (Dev 1 builds this) catches all thrown errors. You do not write res.status(500) anywhere except in the error handler.
- For validation errors (CSV, request body): return 400 with errors array before any DB writes.

```
// Correct pattern for all controllers
const asyncHandler = fn => (req, res, next) => Promise.resolve(fn(req, res,
next)).catch(next);

router.post('/some-route', verifyToken, requireRole('teacher'), asyncHandler(async (req,
res) => {
  // your logic — throw CustomError on validation/business logic failures
  // never catch errors here — let them bubble to global handler
}));
```

Validators

- Every endpoint that accepts a request body must have a validator in src/validator/.
- Use express-validator.
- Validators run as middleware before the controller.
- Validator rejects malformed requests before they reach service or DB layer.

Logging

- Use the apiLogger.js utility (already in utils/) for request/response logging.
- Use errorLogger.js for logging errors. Do not use console.log in production code.

Environment Variables

- Never hardcode URLs, secrets, or config values. All config comes from process.env.
- If you add a new env var, add it to .env.example immediately and announce in group chat.

What You Can and Cannot Touch

File	Rule
constant.js	READ ONLY. Never write to this. Raise with project lead if a value needs changing.
enums.js	READ ONLY. Never modify without project lead approval.
customError.js	READ ONLY. Import and use. Never modify.
markService.js	Only Dev 1 writes this. Everyone else imports and calls functions from it.
cacheService.js	Only Dev 5 writes this. Dev 1 imports invalidateStudentCache().
notificationService.js	Only Dev 2 writes this. Dev 3 and Dev 4 import and call it.
metricsService.js	Only Dev 5 writes this. Dev 3 imports getAllMetrics() for PDF.
database.js	Only project lead edits this. Never touch connection logic.

9 Field naming changes

1. No single `User` collection with a `role` field

The handbook told Dev 1 to put all users (admin, teacher, student) in one `users` collection with a `role` field. Your schema separates them:

- `user` → base identity table
- `student` → extends user with its own profile fields (dob, batchId, streaks, totalPoints, etc.)
- `instructor` → extends user with rating fields

Impact on Dev 1: The `User` model is just the base. Dev 1 needs to also create a `Student` model and an `Instructor` model. The `requireRole()` middleware needs to resolve role via `user.roleId → role.name` instead of `user.role` directly.

Impact on Dev 3 (Student): The student dashboard reads from the `student` table, not `user`. Fields like `totalPoints`, `currentStreak`, `batchId`, `gitHubUrl` live in `student`, not `user`.

---

2. `instructor` replaces `teacher` everywhere

The handbook used the word "teacher" throughout — routes, file names, controller names. Your schema uses `instructor`. This is a naming conflict that will cause confusion if not clarified now.

Decision you need to make and communicate: Are they the same thing? From context, yes — instructor = teacher in the handbook. Tell the team: use `instructor` in all model/variable names to match the schema.

Affected devs: Dev 1 (model name), Dev 2 (all controller/service/route file names), Dev 4 (csvController references teacher routes).

---

3. `session` replaces `lecture`

Your schema uses `session` (not `lecture`). The handbook used `lecture` everywhere. Same concept, different name.

`session` has some extra fields the handbook didn't mention: `recordingUrl`, `duration`, `status enum includes 'live' and 'postponed'` (handbook only had scheduled/in_progress/completed/cancelled).

Impact on Dev 2: Model is named `Session`, not `Lecture`. Status enum needs `'live'` and `'postponed'` added. File names: `sessionController.js`, `sessionService.js`, `sessionRoutes.js`.

Impact on Dev 4: CSV upload routes reference `sessionId`, not `lectureId`.

---

4. `course` is a new layer the handbook didn't have

Your schema introduces a `course` table between batch and session. A course has instructors, and sessions belong to a course. Students enroll in courses (`enrolledCourseIds` on student).

The handbook assumed: **batch → lecture directly**.
Your schema says: **batch → course → session**.

This affects:

- **Dev 2** needs to build Course CRUD in addition to Session management.
- **Dev 4's** attendance table now has `courseId` as well as `sessionId` — both need to be present in the CSV processing logic.
- **Dev 5's** metrics need to be scoped by course, not just batch, for some metrics (e.g. `sessionAttended / totalSession` per course).

---

5. `assignmentResult` is a separate table from `assignmentSubmission`

The handbook merged submission + review result into one `Submission` document. Your schema separates them cleanly:

- `assignmentSubmission` → what the student submitted (GitHub link, repo, branch, remarks, onTimeSubmission)
- `assignmentResult` → what the code review returns (marks, codeQualityScore, feedback, evalBy, result pass/fail, bonusPoints)

Impact on Dev 3: `Submission` model maps to `assignmentSubmission`. After code review completes (Dev 2's queue), a separate `AssignmentResult` document is created — not an update to the submission.

Impact on Dev 2: The code review queue worker creates an `AssignmentResult` document on completion, then writes to the `studentLedger`.

---

6. `studentLedger` uses `points`, not `marksApplied`

The handbook called the scoring unit "marks" throughout. Your schema uses `points` (`studentLedger.points`, `assignmentResult.points`, `quizResult.points`, `student.totalPoints`).

This is a naming consistency issue. The handbook's `markService.js` should be renamed `pointsService.js` or `ledgerService.js`, and all references to `marksApplied` become `points`.

Affects every dev. Communicate this clearly before anyone writes a line of code.

---

7. `studentMetrics` is a stored table, not computed at query time

The handbook told Dev 5 to compute all metrics at query time from the ledger (with Redis caching). Your schema has a `studentMetrics` table — meaning metrics are **pre-computed and stored**, not derived on the fly.

This is a fundamental architectural difference. Two options — you need to decide:

- **Option A (follow schema):** Metrics are written to `studentMetrics` table whenever a new ledger event occurs. Dev 5 reads from this table instead of computing live. Faster reads, but requires a trigger/job to keep the table updated.
- **Option B (follow handbook):** Drop the `studentMetrics` table, compute at query time with Redis cache. Simpler code, slightly slower on cache miss.

Recommendation: Follow the schema (Option A). It's more scalable and the schema was clearly designed with this in mind. Dev 5's job becomes: a) update `studentMetrics` whenever `studentLedger` gets a new entry, b) serve recruiter endpoint from `studentMetrics` directly.

---

8. `quizResult` has no `reviewStatus`/`error` handling

The handbook had a quiz CSV upload flow with validation errors. Your `quizResult` table stores the final result directly with no pending/error state. This is fine — just means Dev 4's quiz processing writes directly to `quizResult` after validation passes. No interim status needed.

---

9. `attendance.status` enum differs

Handbook had: `both / 1 / 2 / none` (for half-attendance tracking via CSV).
Your schema has: `present / absent / leave / half`.

The attendance mark rules from the SRS still apply, but the CSV `half_attended` column values need to map to your enum:

CSV value	Maps to attendance.status	Marks
`both`	`present`	+2.5
`1` or `2`	`half`	+1.0
`none`	`absent`	−5.0

Dev 4 needs to handle this mapping explicitly in the CSV processing logic.

Summary Table

Area	Handbook Said	Schema Says	Action Required
User model	Single collection, role field	`user` + `student` + `instructor` separate tables	Dev 1 builds 3 models
Teacher naming	"teacher" throughout	`instructor`	Rename everywhere
Lecture naming	"lecture" throughout	`session`	Rename everywhere
Lecture status enum	scheduled/in_progress/completed/cancelled	adds `live` and `postponed`	Dev 2 updates enum

Batch→Session	Direct	Via `course` layer	Dev 2 adds Course CRUD
Submission + review	One document	Two tables (`assignmentSubmission` + `assignmentResult`)	Dev 2 + Dev 3 split models
Scoring unit	"marks" / `marksApplied`	`points`	Rename everywhere, all devs
Metrics	Computed at query time	Stored in `studentMetrics` table	Dev 5 rewrites approach
Attendance enum	both/1/2/none	present/absent/leave/half	Dev 4 adds mapping layer
`markService.js` name	`markService`	Should be `ledgerService` or `pointsService`	Dev 1 renames

— End of Document —